

Reto 1. Crear y listar elementos

[Creación de un formulario para añadir elementos](#)

[Visualización de la lista de la compra](#)

[Implementación de un botón para eliminar elementos](#)

[Refactorización del componente](#)

Componente `ShoppingList`

Componente `ShoppingListItems`

Componente `ShoppingListForm`

Creación de un formulario para añadir elementos

En este apartado, vamos a detallar cómo desarrollar un formulario en React para la creación de elementos de la lista de la compra.

Cada elemento de la lista de la compra será un objeto que incluye tres propiedades: producto, cantidad y fecha de compra.

Nuestro objetivo es añadir estos elementos a un arreglo una vez que el formulario sea enviado.

Para comenzar, definamos la estructura básica de nuestro componente de React. Este componente manejará el estado del formulario y la lista de elementos de la lista de la compra.

```
import { useState } from 'react';

function ShoppingListForm() {
  const [shoppingItems, setShoppingItems] = useState([]);
  const [newItem, setNewItem] = useState({
    product: '',
    quantity: '',
    purchaseDate: ''
  });

  function handleInputChange(event) {
    const { name, value } = event.target;
    setNewItem(prevItem => ({
      ...prevItem,
      [event.target.name]: event.target.value
    }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    setShoppingItems(prevItems => [...prevItems, newItem]);
    setNewItem({ product: '', quantity: '', purchaseDate: '' });
  }

  return (
    <form onSubmit={handleSubmit}>
      <p>
        <label htmlFor="product">Producto:</label>
      </p>
      <p>
        <input
          id="product"
          name="product"
          value={newItem.product}
          onChange={handleChange}
        />
      </p>
    </form>
  );
}
```

```

        <label htmlFor="quantity">Cantidad:</label>
      </p>
      <p>
        <input
          id="quantity"
          name="quantity"
          value={newItem.quantity}
          onChange={handleChange}
        />
      </p>
      <p>
        <label htmlFor="purchaseDate">Fecha de compra:</label>
      </p>
      <p>
        <input
          id="purchaseDate"
          name="purchaseDate"
          value={newItem.purchaseDate}
          onChange={handleChange}
        />
      </p>
      <div>
        <button type="submit">Guardar</button>
      </div>
    </form>
  );
}

export default ShoppingListForm;

```

En este código, `ShoppingListForm` es un componente funcional que utiliza el hook `useState` para gestionar el estado. Mantenemos dos estados: `shoppingItems`, que es un arreglo de elementos de la lista de la compra, y `newItem`, que guarda los valores actuales del formulario.

Para cada campo del formulario (producto, cantidad, fecha de compra), utilizamos un control de entrada (`input`) con un `name` correspondiente. El atributo `value` de cada control se vincula al estado `newItem`, asegurando que el estado siempre refleje lo que el usuario ha ingresado.

La función `handleInputChange` se dispara en cada cambio en los campos del formulario, actualizando el estado `newItem` con los valores actuales. Utilizamos la desestructuración para obtener el nombre y el valor del campo que ha desencadenado el evento.

Al enviar el formulario (evento `onSubmit` del `form`), la función `handleSubmit` es llamada. Esta función previene el comportamiento por defecto del formulario, añade el nuevo elemento de la lista de la compra al arreglo `shoppingItems` y reinicia los valores del formulario a su estado inicial.

Con este enfoque, hemos establecido un formulario funcional que permite a los usuarios ingresar datos para un nuevo elemento de la lista de la compra y, tras enviar el formulario, añadir este elemento a una lista almacenada en el estado del componente. Es importante mencionar que en este ejemplo solo estamos manejando el estado dentro del componente y no mostramos la lista de elementos de la lista de la compra. En una aplicación real, es posible que quieras pasar la lista de elementos de la lista de la compra a otros componentes o incluso elevar el estado para manejarlo en un nivel superior de tu aplicación.

Visualización de la lista de la compra

Continuando con nuestro componente `ShoppingListForm`, ahora vamos a incorporar la funcionalidad para visualizar la lista de elementos de la lista de la compra. Cada elemento de la lista de la compra, como recordarás, es un objeto con tres propiedades: producto, cantidad y fecha de compra. Vamos a mostrar estos elementos en una lista debajo del formulario.

Para lograr esto, añadiremos en nuestro componente una sección para renderizar la lista de elementos de la lista de la compra. Cada elemento se mostrará en un formato estructurado para facilitar la lectura. A continuación, el código actualizado del componente:

```

import { useState } from 'react';

function ShoppingListForm() {
  const [shoppingItems, setShoppingItems] = useState([]);
  const [newItem, setNewItem] = useState({
    product: '',
    quantity: '',
    purchaseDate: ''
  });

  function handleInputChange(event) {
    const { name, value } = event.target;
    setNewItem(prevItem => ({
      ...prevItem,
      [name]: value
    }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    setShoppingItems(prevItems => [...prevItems, newItem]);
    setNewItem({ product: '', quantity: '', purchaseDate: '' });
  }

  return (
    <div>
      <form onSubmit={handleSubmit}>
        /* Campos del formulario aquí */
      </form>
      <div>
        {shoppingItems.map((item, index) => (
          <div key={index}>
            <h3>{item.product}</h3>
            <p>Cantidad: {item.quantity}</p>
            <p>Fecha de compra: {item.purchaseDate}</p>
          </div>
        ))}
      </div>
    </div>
  );
}

export default ShoppingListForm;

```

Dentro del componente `ShoppingListForm`, después del formulario, hemos añadido un nuevo bloque `div` que se encarga de renderizar la lista de elementos de la lista de compra. Utilizamos el método `map` para iterar sobre el arreglo `shoppingItems`. Cada elemento se muestra en un `div` individual, donde presentamos el producto, la cantidad y la fecha de compra. Es importante asignar un `key` único a cada elemento de la lista para que React pueda manejar eficientemente la actualización y renderizado de la lista.

En este bloque, cada elemento es un `div` que contiene un `h3` para el producto y dos párrafos (`p`) para la cantidad y la fecha de compra, respectivamente. Esto asegura que la información de cada elemento se presente de manera clara y estructurada.

Con esta adición, nuestro componente `ShoppingListForm` no solo permite a los usuarios ingresar y enviar elementos de la lista de la compra, sino que también muestra estos elementos en una lista debajo del formulario. Esta es una manera sencilla y efectiva de visualizar los elementos de la lista de la compra que han sido agregados, mejorando la interactividad y funcionalidad de nuestra aplicación React.

Implementación de un botón para eliminar elementos

Proseguimos con la mejora de nuestro componente `ShoppingListForm`, incorporando ahora un botón para eliminar elementos de la lista de la compra individualmente. Esta funcionalidad es crucial para permitir a los usuarios gestionar eficazmente su lista de la compra, eliminando aquellos elementos que ya no son necesarios o que han sido comprados.

Para lograr esto, modificaremos la manera en que se muestran los elementos, añadiendo un botón de eliminación a cada uno de ellos. Asimismo, implementaremos una función para manejar el evento de eliminación. A continuación, el código actualizado:

```
import { useState } from 'react';

function ShoppingListForm() {
  const [shoppingItems, setShoppingItems] = useState([]);
  const [newItem, setNewItem] = useState({
    product: '',
    quantity: '',
    purchaseDate: ''
  });

  function handleInputChange(event) {
    const { name, value } = event.target;
    setNewItem(prevItem => ({
      ...prevItem,
      [name]: value
    }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    setShoppingItems(prevItems => [...prevItems, newItem]);
    setNewItem({ product: '', quantity: '', purchaseDate: '' });
  }

  function handleDelete(index) {
    setShoppingItems(prevItems => prevItems.filter((item, i) => i !== index));
  }

  return (
    <div>
      <form onSubmit={handleSubmit}>
        { /* Campos del formulario aquí */ }
      </form>
      <div>
        {shoppingItems.map((item, index) => (
          <div key={index}>
            <h3>{item.product}</h3>
            <p>Cantidad: {item.quantity}</p>
            <p>Fecha de compra: {item.purchaseDate}</p>
            <button
              onClick={() => handleDelete(index)}
            >
              Eliminar elemento
            </button>
          </div>
        ))}
      </div>
    </div>
  );
}
```

```

    );
  }

  export default ShoppingListForm;

```

En cada elemento de la lista de la compra, ahora incluimos un botón con el texto "Eliminar elemento". Este botón está vinculado a un evento `onClick` que llama a la función `handleDelete`, pasando el índice del elemento como argumento.

La función `handleDelete` es crucial aquí. Lo que hace es actualizar el estado `shoppingItems`, eliminando el elemento seleccionado. Esto lo logra mediante el método `filter`, creando un nuevo arreglo que contiene todos los elementos excepto aquel cuyo índice coincida con el proporcionado. De esta forma, el elemento seleccionado se elimina de la lista.

Esta implementación ofrece una interfaz intuitiva para los usuarios, permitiéndoles no solo añadir elementos a la lista de la compra, sino también eliminarlos individualmente según sea necesario. Es un paso fundamental para mejorar la gestión de la lista de la compra en nuestra aplicación React, haciéndola más dinámica y adaptativa a las necesidades del usuario.

Refactorización del componente

Para optimizar nuestro código y seguir las mejores prácticas de React, vamos a refactorizar nuestro componente de elementos de la lista de la compra en tres componentes separados: `ShoppingList`, `ShoppingListItems` y `ShoppingListForm`. Esta estructura modular nos permitirá una mejor organización del código y facilitará la mantenibilidad y escalabilidad de nuestra aplicación.

Componente `ShoppingList`

El componente `ShoppingList` actuará como el componente principal. Contendrá el estado `shoppingItems` y las funciones `handleSubmit` y `handleDelete`. Además, será responsable de renderizar los componentes `ShoppingListItems` y `ShoppingListForm`.

```

import { useState } from 'react';
import ShoppingListForm from './ShoppingListForm';
import ShoppingListItems from './ShoppingListItems';

function ShoppingList() {
  const [shoppingItems, setShoppingItems] = useState([]);

  function handleSubmit(newItem) {
    setShoppingItems(prevItems => [...prevItems, newItem]);
  }

  function handleDelete(itemIndex) {
    setShoppingItems(prevItems => prevItems.filter((_, index) => index !== itemIndex));
  }

  return (
    <div>
      <ShoppingListForm onSubmit={handleSubmit} />
      <ShoppingListItems
        shoppingItems={shoppingItems}
        onDelete={handleDelete}
      />
    </div>
  );
}

export default ShoppingList;

```

Componente `ShoppingListItems`

`ShoppingListItems` se encargará de mostrar la lista de elementos de la lista de la compra. Recibirá la lista de elementos y la función para eliminar elementos como props.

```
function ShoppingListItems({ shoppingItems, onDelete }) {
  return (
    <div>
      {shoppingItems.map((item, index) => (
        <div key={index}>
          <h3>{item.product}</h3>
          <p>Cantidad: {item.quantity}</p>
          <p>Fecha de compra: {item.purchaseDate}</p>
          <button onClick={() => onDelete(index)}>Eliminar elemento</button>
        </div>
      ))}
    </div>
  );
}

export default ShoppingListItems;
```

Componente **ShoppingListForm**

El componente **ShoppingListForm** será el encargado de renderizar el formulario. Mantendrá su propio estado local para los valores del formulario y recibirá la función **onSubmit** del componente **ShoppingList**.

```
import React, { useState } from 'react';

function ShoppingListForm({ onSubmit }) {
  const [newItem, setNewItem] = useState({
    product: '',
    quantity: '',
    purchaseDate: ''
  });

  function handleInputChange(event) {
    const { name, value } = event.target;
    setNewItem(prevItem => ({
      ...prevItem,
      [name]: value
    }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    onSubmit(newItem);
    setNewItem({ product: '', quantity: '', purchaseDate: '' });
  }

  return (
    <form onSubmit={handleSubmit}>
      <p>
        <label htmlFor="product">Producto:</label>
      </p>
      <p>
        <input
          id="product"
          name="product"
          value={newItem.product}
          onChange={handleInputChange}
        >
      </p>
    </form>
  );
}
```

```

    />
  </p>
  <p>
    <label htmlFor="quantity">Cantidad:</label>
  </p>
  <p>
    <input
      id="quantity"
      name="quantity"
      value={newItem.quantity}
      onChange={handleInputChange}
    />
  </p>
  <p>
    <label htmlFor="purchaseDate">Fecha de compra:</label>
  </p>
  <p>
    <input
      id="purchaseDate"
      name="purchaseDate"
      value={newItem.purchaseDate}
      onChange={handleInputChange}
    />
  </p>
  <div>
    <button type="submit">Guardar</button>
  </div>
</form>
);
}

export default ShoppingListForm;

```

Con esta refactorización, hemos dividido el componente original en tres partes más pequeñas y manejables. Cada componente ahora tiene una responsabilidad clara y definida, lo que mejora la legibilidad del código y facilita futuras modificaciones o ampliaciones. Además, esta estructura facilita la reutilización de componentes en otras partes de la aplicación si fuera necesario.